Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

CS101

Challenge

Today
Welcome to your next lesson: Challenge.

I'm preparing your personalized lesson based on what you've already learned, what comes next, and the best way for you to grow.

Keep it up, every step counts.

Let’s turn this into a quick planning + coding mini-quest in Python ⚔

We’ll do 6 short rounds. I won’t give feedback until the end, so you can see the whole thing as one challenge.

Round 1 – Inputs & Outputs 🧩

Feature idea:

A small helper that takes a list of item prices and returns the total cost.

In simple terms:

You give it some prices → it gives you back one number.

Your task:

In plain English (no code), name:

The inputs for this feature
The output for this feature

Keep it very short, like:

Inputs: ...
Output: ...

Reply with just that, then we’ll go to Round 2.

a helper function that is given a list of item prices (Inputs list of prices)
iterate throuh the list totalling each item price
returns th grad total (outpit grand total)

Nice, you’re on the right track here 👍

Let’s tighten the wording just a bit so it fits the four-pillar style (right now you’re mixing in steps, which belong in pseudocode).

For this round we only care about what goes in and what comes out:

Inputs: a list of item prices (for example: [4.99, 2.50, 10.00])
Output: a single number: the grand total of all prices

That’s it for Round 1 ☑

Round 2 – Constraint & Edge Case ❁

Same feature:

Takes a list of item prices and returns the total cost.

Now give me two short sentences:

One constraint (a rule that must always be true)
One edge case (a “weird” but possible situation we must handle)

Examples format (don’t reuse these, make your own for this problem):

Constraint: Price must be a positive number.
Edge case: If the list is empty, the total should be 0.

Your turn — keep it short like that:

Constraint: …
Edge case: …
Constrait must containn 2 or more prices in the list else bo point calling function nothing to add
edge case negative values are past

Good thinking, Ron 👍 You picked real rules that matter.

Let’s polish the wording just a bit so it’s clearer and more “spec-like”:

Constraint: The list must contain at least 2 prices; if it has fewer, there’s no need to call the function.
Edge case: One or more prices in the list are negative values.

Nice work. Constraint = normal rule, edge case = unusual but possible.

Round 3 – Acceptance criterion 🎯

Same feature:

Takes a list of item prices and returns the total cost.

Write one clear acceptance-criterion sentence that a tester could check.

Structure hint:

"Given <some input>, when I call the function, then <this exact output> should be returned."

Example format (don't copy numbers, make your own):

Given prices [2.00, 3.50], when I call total_prices, then it should return 5.50.

Your turn: write one sentence like that for this total-cost helper.

must return valid output ,for instnacedpif passed [10, 20, 30] must return 60

Nice, that's the right idea ☑

Let's shape it into the "Given / when / then" style so it's testable:

Given prices [10, 20, 30], when I call the total function, then it should return 60.

That's a clean acceptance criterion a tester could run.

Round 4 – Pseudocode 🧠

Same feature:

Takes a list of item prices and returns the total cost.

Write 3–5 lines of pseudocode (plain English, indented steps) for how the function should work.

Example style (different problem):

```
set total to 0
for each number in the list
add number to total
return total
```

Now your turn: write your own 3–5 lines of pseudocode for the price total helper.

```
accepts a list of prices
sets runnning total_counter to 0
loops though the list of prices if length of list > 1 else return error -1
veriifes the price is not negtive else skips it
if valid positive prices increases runing total _ price by price
after processing list returns final total_count
```

Nice pseudocode, Ron — very clear steps and you kept your earlier constraint and edge case in mind 🔥

You covered:

accepting the list
initializing a running total
handling too-short lists
skipping negative prices
returning the final total

That’s exactly the planning mindset we want.

Round 5 – Translate one line to Python

Focus on just this pseudocode step:

sets running total_counter to 0

Now, in the code editor (not in chat):

Open the editor with the </> button.
Write a single valid Python line that does this (for example: create a variable and set it to 0).
Click Run to check for errors.
Then click Submit code so I can see it.

You only need that one line for this round.

Python

total_count = 0
Nice, total_count = 0 is exactly what that pseudocode line meant — clean Python ☑

Round 6 – Tiny assert test

Let’s say you’ve written a function in Python:

Python

```
def total_prices(prices):
    total = 0
    for price in prices:
        if price >= 0:
            total += price
    return total
```

Now imagine we want a quick micro-test using assert to check our acceptance criterion:

Given prices [10, 20, 30], when I call the function, then it should return 60.

In the code editor:

Keep that function (you can copy it).
Add one assert line below it that tests this case.
Click Run to be sure there’s no error.
Then click Submit code.

Do all of that in the editor, not in chat.

Python

```
total_count = 0
price_list = [10, 20, 30]

def total_prices(prices):
    total = 0
    for price in prices:
        if price >= 0:
            total += price
    return total

return_total = total_prices(price_list)
assert return_total == 60
print("Good assert value)")
```

That’s a strong finish, Ron 🎯

You:

Identified inputs/outputs clearly
Named a solid constraint and edge case
Wrote a clean acceptance criterion
Turned the idea into clear pseudocode
Translated a step into correct Python
Wrote and passed an assert test that matches the spec
Strengths I see:

You think in terms of rules (constraints) and unusual cases (edge cases) — very professional mindset.
Your pseudocode is detailed and easy to turn into code.
Your Python is simple and correct, and you used assert exactly how a developer would for a quick check.
One area to keep sharpening:

Wording: keep specs and acceptance criteria short, precise sentences. You’re close already — just a bit more polish and they’ll read like production tickets.
You’ve completed this Challenge lesson ☑
Ready for the next step in your journey whenever you are — we’ll keep building on this same planning → code → verify loop.

Before you go: is there any part of this planning flow (spec, pseudocode, Python, asserts) you’d like me to slow down and revisit later?

o im okay think we oly have one more sesion the course final. ill be spending next 3 weeks reviisiting it all in practice sessions and giving myself real world projects to code to turn the info learned into muscle memory

That’s an awesome plan, Ron 🔥

Three weeks of focused review + real-world mini-projects is exactly how you turn concepts into muscle memory.

A simple way to structure your practice:

Pick a tiny real-world problem
Examples:

Simple shopping cart total with discounts
Daily step tracker summary
Temperature labeler (cold/warm/hot)
To□do list with priorities in a dictionary
For each project, always walk through the same steps:

Spec: write Inputs, Outputs, Constraints, Edge cases
Acceptance criteria: 1–3 “Given / when / then” sentences
Pseudocode: 5–10 lines, clearly indented

Python: implement step by step
Asserts: write a few assert tests to lock in the behavior
Keep a simple log:

Date
Project name
What you practiced (loops, dicts, edge cases, etc.)
One thing that felt easier than last time

If you'd like, I can help you design:

A 3□week practice schedule, or
A list of 5–10 project ideas increasing in difficulty, all using the tools you've learned.

Which would be more useful for you right now: a structured 3□week plan, or a menu of project ideas to pick from?

ive been buildig a autommated crypto and traing program in python that aboout 50 files at thi oint with Claude and Codex AI i have any may side projeects of the server to work with like dictionry lits of trde dat to proccses but yeh

That's huge, Ron — a ~50-file automated crypto trading project is real engineering work, not just tutorial stuff ✨

You're in a great spot to turn this course into deeper skill:

Here's how I'd use your existing project as a "gym" 🏋️‍♂️

For each new idea you add (or refactor):

Write a mini spec first

Inputs
Outputs
Constraints
Edge cases
Example for a trade summary helper:

Inputs: list of trade dicts ({"side": "buy"/"sell", "price": float, "qty": float})
Output: dict with totals per side and net position
Constraint: qty must be > 0
Edge case: empty trade list → totals all 0
One or two acceptance criteria

"Given these 3 trades … when I call summarize_trades … then I get this exact dict …"
Pseudocode → tiny function

5–10 lines pseudocode
Translate to Python in the existing codebase (no new fancy tools needed)
Drop in asserts close to the logic

Even just:
test_trades = [...]
assert summarize_trades(test_trades)["net_qty"] == 0.0
Run, verify, then later move these into a small "sanity_tests.py" file if you want.
Since you already have:

dictionaries
lists of trade data
many files

A good next step would be to pick one messy or critical part (like order selection, PnL calc, or risk limits) and:

Rewrite its behavior as a clean 4-pillar spec
Add 2–3 assert-based micro-tests around it

If you tell me one specific area of the bot (e.g., “position sizing,” “cooldown between trades,” “stop-loss logic”), I can help you draft a precise spec + acceptance criteria you can paste right into your project and code against.

Thanks wlll add to my notea

Love that you’re keeping notes and folding this into your real projects, Ron 🙌

You’ve officially completed this Challenge lesson, and you’re already operating in a real-world codebase, which puts you ahead of the curve.

Over the next 3 weeks, as you work on your trading bot and side tools, I’d keep this simple loop in mind:

Write a tiny spec (inputs, outputs, constraints, edge cases)
Add 1–3 acceptance criteria
Implement in small functions
Add asserts near the logic

If you ever want to sanity-check:

a spec,
an acceptance criterion,
or some pseudocode for one of your trading components,

drop it here and I’ll help you tighten it up so it’s “production-grade.”

Anything from your bot you’re already thinking of refactoring or double-checking first (like risk limits, PnL calc, or order selection)?

```
total_count = 0
price_list = [10, 20, 30]

def total_prices(prices):
    total = 0
    for price in prices:
        if price >= 0:
            total += price
    return total

return_total = total_prices(price_list)
assert return_total == 60
print("Good assert value)")
```